

GEM Data Submission Guidelines

GEM Data Submission Guidelines.....	1
Introduction.....	1
GitHub.....	2
Repository Structure.....	2
Daily workflow.....	3
Before field sampling.....	3
Field steps.....	3
Post-field steps - Survey.....	3
Post-field steps - Instruments.....	4
Lab analysis steps.....	4
Data processing.....	4
GitHub Concepts - the 'basics'.....	5
Branches.....	5
Commit.....	5
Pull request.....	5
GitHub Interactions - the 'basics'.....	5
Creating folders.....	5
Uploading data.....	6
Calibration Files.....	7
Failed Checks.....	8
Issues.....	8
Example Survey Day Workflow.....	8

Introduction

This document provides data submission guidelines for the participants of the Global Eutrophication Monitoring (GEM) initiative. The guidelines will be useful for people that are not familiar with GitHub or know how to interact with it, and it will cover the basics of GitHub interactions and features.

Should you have any questions or concerns about the data submission process, please reach out to me at tim.vanderstap@hakai.org, or create an issue (see [here](#)). You can assign me (@timvdstap) and I will get a notification. Please be as descriptive as you can about your problem so I can best help you address it.

GitHub

GitHub is used for the data management and submission of the GEM project. [GitHub](#) is an open source, cloud-based, version-control platform that offers excellent storing, management and collaboration capabilities for plain text files and code. Create a free account at github.com. You will receive an invitation from us to collaborate on a repository.

A unique GitHub repository is created for, and shared with, each project participant. A repository is essentially a cloud-based data folder where you can upload your data to, along with any supporting documentation such as sampling protocols. The individual GitHub repositories are created by the Hakai Institute, following a specific template to ensure that the contents and folder structure of each repository are uniform across all participants. The repositories will initially be private, meaning that they will not be visible and accessible to the general public. As has been described in the [Data Management Plan](#), the repositories and the data included therein will not be made public until December 31, 2025. All project participants will have permissions to manage their respective repositories and upload data to it.

Repository Structure

Across all participants, each repository contains the same folders and files that will need to be populated or modified throughout the project. Each repository and folder included contains a readme file that outlines what the contents are. Each repository contains the following folders:

- Blank data sheets: includes blank data sheets that you can either bring with you into the field (stored in the subfolder: Field Sheets) or that you will need to fill out after lab analysis or coming back from the field (Data Sheet Templates). These field and data sheets are standardized, which will make it easier to compare, integrate and serve up the data at a later stage of the project.
- Calibration tracking: includes two files for the AquaFluor where you can keep track of the calibration and performance of the instrument.
- Data: this is arguably the most important folder and the one that you will be interacting with most. This is the folder you will upload all your data to.
- The docs/logos folder contains .png files for the logos you will see in the repository. There is no need to interact with this folder.
- Protocols: includes all the relevant protocols for the GEM project, including a subfolder 'GitHub how-to' for additional written documentation on how to interact with GitHub.
- Safety data sheets: relevant safety data sheets for reference.
- Finally, there's the scripts folder - if you are familiar with programming languages such as R or Python, you can include any relevant scripts for data processing or cleaning in this folder.

Additionally, there are a few files within the root of your repository (i.e., the main page) that are worth highlighting:

- Changelog: When you release, or make public, multiple versions of your data package or repository, it is strongly recommended to include any additions, changes, or deletions between versions in a changelog.
- License: Outlines the terms and conditions under which data may be used, modified or shared by downstream users. As is mentioned in the Data Management Plan, we strongly encourage data to be shared under the [CC-BY-4.0 license](#).
- Readme: You can find the readme in the root of the repository, at the bottom. It includes the title, a paragraph on how to get started, additional headings to describe the methods used, and links to any reports and external resources.
- Data-dictionary: provides a description for each column of the various field and data sheets used. Unless you are adding columns to your field or data sheets, you will not have to make any changes to this file.
- Project-metadata: contains metadata specific to each project participant, such as kit number, contact person's name, ORCID (if applicable), and email. This file will need to be updated by each project participant.
- stations.csv: contains the latitude and longitude, in decimal degrees, of each sampling station. Please note that when you do the actual data sampling, the coordinates might be slightly off from your station coordinates, so you will still have to record latitude and longitude in decimal degrees for each sampling event.

Daily workflow

I have outlined at a high level the suggested daily workflow in [this issue](#), which you can find at the top of the webpage under the 'Issues' tab. Below, I have expanded on the daily workflow steps.

Before field sampling

Download the survey sheet and the instrument data sheet templates to your computer or laptop, and print them.

Field steps

On a survey, you should fill out a paper copy of the survey data sheet. When you arrive at your sampling site, follow the field sampling protocols and collect your data. Populate the rest of the survey and instrument data sheets accordingly, confirm that all data is collected before moving on to the next station.

Post-field steps - Survey

Take a picture of your survey template sheet that you filled out in the field, save it on your local device as a **.jpeg** file following the naming convention 'YYYY-MM-DD_survey_raw.jpeg': a four-digit year, followed by a hyphen, 2 digit month, followed by a hyphen, and then 2 digit day, followed by _survey_raw. Next, if you have not already done so, download the survey template sheet, which you can find [here](#). Transcribe your physical survey sheet onto that template, saving it as a **.csv** file: YYYY-MM-DD_survey_final.csv.

If you visited a new station, make sure you update the information in your **stations.csv** file as well, which can be found in the root of your repository. You can update it by downloading it to your local device, making the change, and then re-uploading it.

Post-field steps - Instruments

When returning from the field, download the data from the PME miniDOT and CTD-diver, saving the files on your local device in the original format. Please rename the raw data files from the instrument again following the convention YYYY-MM-DD_[Instrument]_raw, in the original file format (for example: 2024-09-16_miniDOT_raw). Transcribe the data from the original file onto the instrument template sheet, saving it as YYYY-MM-DD_[Instrument]_processed, which can be either a .csv file or a .xlsx file. Conduct quality control, recording any modifications, removals or additions between the original instrument and the processed data file in a Changelog.txt file. Finally, save the final processed file as YYYY-MM-DD_[Instrument]_final.csv.

Lab analysis steps

Before running the lab analyses through the Aquafluor and DR1900, download the Aquafluor and DR1900 data sheet templates and print them. Analyze the water samples following the protocols, recording the data on those data sheets. Next, take a picture of the physical Aquafluor and DR1900 data sheets, saving it locally as for example YYYY-MM-DD_Aquafluor_raw_datasheet1.jpeg. Any additional pictures of datasheets would be YYYY-MM-DD_Aquafluor_raw_datasheet2.jpeg, etc. (same will apply for the DR1900). Transcribe the data from the physical data sheet onto the Aquafluor and DR1900 data sheets, performing an initial round of quality control. Record modifications, additions and removals in the Changelog.txt file. Save those files locally, again following YYYY-MM-DD_[Instrument]_processed, e.g., 2024-11-27_Aquafluor_processed.csv. Conduct another round of quality control before saving the final processed file as YYYY-MM-DD_[Instrument]_final.csv.

Data processing

Each sampling day will have its own folder that will contain the data from that day. Create a new survey-specific folder within the data folder (see [creating folders](#)), following the naming convention YYYY-MM-DD_survey_data. [Commit](#) this new folder to a separate [branch](#), and open a [pull request](#). Within that branch, create the subfolders for Aquafluor, DR1900 and Instrument data within the survey data folder. Before uploading the data to the folders, make sure you are in the correct branch. Upload the raw and processed **survey** sheets into the root of the survey folder. Next, upload the raw, processed and final **data** files from the instrument and lab analyses, along with changelog files where relevant, to the appropriate folders.

Once you have uploaded all the survey and sampling event data, and the checks within the pull request are successful, merge the branch with the main branch.

GitHub Concepts - the 'basics'

To interact with GitHub there are a few important things to familiarize yourself with: editing, creating folders, and downloading and uploading files, i.e. the "GitHub basics". The 'how to' for these can be found [here](#).

Branches

In GitHub you can create *additional* branches. You will notice that the *main* branch is protected, meaning that you cannot make changes directly to it. When you make changes, you will have to save ([commit](#)) these to a *separate* branch, which runs parallel to the main branch. This allows you to make changes without changing the main branch. Once you are satisfied with your changes, you can submit a [pull request](#).

Commit

To save your changes locally, you have to [commit](#) them in GitHub. You will notice that you cannot make a commit directly to the main branch. The reason for this is so that we can keep the main branch 'clean', and keep track of any commits to a specific survey day within a separate branch. Make your commits to a separate branch before merging the branch with the main branch.

Pull request

A pull request is essentially a request from the main branch to *pull in* the contents or the changes of the branch you created. A pull request can contain multiple commits. Merging changes through a pull request gives GitHub the opportunity to run a few tests and checks on the content you wish to merge, and you can easily review the changes made. Creating (or opening) a pull request [does not mean](#) you are already merging this branch with your main branch. Rather, you are setting up the connection between the two branches.

GitHub Interactions - the 'basics'

Creating folders

Each sampling day will have its own folder that will contain the relevant data from that day within. To do so, create a new survey-specific folder within the data folder, following the naming convention YYYY-MM-DD_survey_data. To do so, follow these steps:

1. Navigate to the 'data' folder, and select 'Add file' and then 'Create new file'.
2. At the top of the page you can name your file. In this case however, you're not naming, or creating, a file but a folder for that specific survey day. To do so, add the name of the folder following the naming convention, and then followed by a forward slash (/). So for example, 2024-12-02_survey_data/. However, you cannot create an empty folder, and so you'll have to add an empty file called [.gitkeep](#) (**there is a . in front of gitkeep**).
3. To then create this folder with the .gitkeep file, you select "Commit changes" in the top right. You'll have to create a new branch, commit your changes there and then start a

pull request. It's useful to name your branch something you'll be able to easily remember and navigate to - e.g., the survey date.

4. Next, you'll be taken to a page where you can create your pull request. Here you can provide a description, for example: "Adding data collected on YYYY-MM-DD". Then, select 'create pull request'.

Note: A new pull request will appear at the top of the page. You'll also see that this is an 'open' branch, and it says that you want to merge a commit into 'main' from this new branch you created. Finally, you'll see four tabs: "Conversation", "Commits", "Checks", and "Files Changed". Particularly the 'Conversation' tab is helpful, if you are running into any issues during your data submission phase you can always tag me (@timvdstap) which will send me a notification and I'll be able to help you out. There's also an initial message which you don't have to pay attention to currently.

5. Finally, navigate back to the landing page, or the root, of your repository by clicking 'Code' in the top left corner.

Important: When you navigate back to the landing page, you'll automatically be directed to the 'main' branch. You will have to make sure you navigate to the branch you have just created. Once you've done that, and you click on the 'data' folder, you should see the folder you just created! When you click on that, it should contain only a single file - .gitkeep.

6. Next, you will have to create the 3 subfolders within this folder, named "Aquafluor", "DR1900" and "Instruments". You do so by navigating to the contents of your newly created survey_data folder, and then again selecting 'Add file' - 'Create new file': similar approach to creating the initial folder. **Unfortunately it is not possible to create 3 new folders all in one go, so you will have to commit these changes for each folder you create.**

Eventually, the structure of your folder should look exactly like the structure of the ["2024-09-16_example_dataset"](#) folder.

Uploading data

1. First, make sure you are on the correct branch – the new branch you created for this survey day! You can see that in the top left corner.
2. Next, let's start with uploading the field survey data. You should have a picture of your physical field data sheet, saved on your local device as a .jpeg file, as well as the template .csv file that has been populated. As a reminder, the physical data sheet and the survey datasheet template file can be found in the ["blank-data-sheets"](#) folder.
3. To upload these files, navigate to the survey date folder you created. As it stands, this just contains 3 empty folders (Aquafluor, DR1900 and Instruments) and the .gitkeep file. Uploading files is a lot more straightforward than creating folders! Click 'add file' and then 'upload files'
4. You can drag the survey files here or choose the files that you wish to upload.

5. You can commit directly to the new branch you created.
6. Now, if you navigate to the survey date folder under 'data', you should be able to see two files in addition to the 3 folders. Nice work!
7. In a similar way, you can upload your instrument data files to the Aquafluor, DR1900 and Instruments folders, respectively. You can look at the [2024-09-16_example_dataset](#) folder to see what the contents of each folder should look like.

Note: If you uploaded the wrong file or created a wrong folder, you can delete a file by clicking on the file, and then navigating to the three dots in the top right corner, and then clicking 'delete file'. You will have to commit these changes as well. To delete a folder, or directory, you'll have to go into the folder, click the three dots, and then select 'delete directory'.

Note: Changing the name of a file is a little easier. To do so, you can click a file, and then click on the pen icon in the right hand corner. Hovering over that should read 'edit this file'. Then you can change the file name before committing your changes.

Calibration Files

1. In your newly created branch for that specific survey date, navigate to for example the [Aquafluor Calibration Tracking Sheet.xlsx](#) file. You can download this to your local device by selecting the arrow down icon.
2. You fill out your calibration information on the Excel sheet on your local device, and save it locally. When saving, make sure you simply keep the same file name. If not, what will happen is that you will create numerous calibration tracking files and you will not know which one is necessarily the latest version. Plus, you'll want to have all your tracking information in the same file.
3. Next, you'll want to replace the previous version of the file from the folder. To do so, delete the Aquafluor calibration tracking sheet from the GitHub folder, and then upload the calibration tracking sheet you have just populated.

In a similar way you can update your stations.csv file which is stored in the root of the repository. While this is a .csv file, and therefore technically can be edited directly through GitHub in the browser, it is likely easier to follow the same steps as described above, replacing the existing file with an updated version.

Once you have done all this, navigate to the pull request you've opened up. If you have named and populated all the files and folders correctly, you should see that all the tests have passed and that if you scroll down, it says: "All checks have passed" and "This branch has no conflicts with the base branch", which is your main branch. What remains is selecting the "Merge pull request". Once this is done, you can safely delete the branch. The contents of this survey day should now be available on the main branch – feel free to check it out!

Failed Checks

We have implemented a few automatic checks - or 'Actions' - within the GitHub repository to ensure that all folders and files are created similarly across all the participants' GitHub repositories – this ensures data and file consistency and will make integration much easier down the road. To check which actions have failed, you can navigate to 'Actions' in the toolbar at the top. Select the latest action that has run, and then navigate to 'Summary'. This should show you the test results, and if you scroll down, provide you with an explanation as to why the test has failed. More often than not, this will be because a file or folder is missing or is not named correctly.

Issues

No reason to worry! If you need a hand uploading your data, or have questions in general, you can reach me at tim.vanderstap@hakai.org. Better yet, you can create an *issue* in your repository and tag me. To do so, navigate to 'issues' in the toolbar at the top of the repository and then select 'New issue' - 'Custom issue template': Get started. Here, you can add a title and a description of your problem. Please be as descriptive as possible. To tag me, you can add me as an assignee, by searching for my username or 'handle': timvdstap. When you're happy with your problem description, you can submit the issue at the bottom. Submitting issues is great because we can keep track of the issues and how they've been resolved.

Example Survey Day Workflow

Let's walk through an example so you can see all the GitHub interactions in practice, as well as a detailed walk-through of uploading data.

1. Navigate to your GitHub repository.
2. Download the survey sheet that we will fill out in the field, along with any data sheets that need to be populated during the lab analysis phase. Save them on your local device in a folder that you can easily remember and access. Print out the sheets.
3. Once you've collected the data in the field and you have filled out all the relevant datasheets – **making sure to have saved them following the naming convention – [create your survey date folder](#)** and the three subfolders.
4. Navigate back to the landing page, or the root, of your repository by clicking 'Code' in the top left corner.
5. Upload your data to the respective folders following the steps outlined in [Uploading data](#).
6. Once all data has been uploaded, navigate to the open pull request
7. Address any [failed checks](#)
8. If there are no failed checks, merge your branch to the main